

Assignment 2: AI-Powered Customer Support Automation System using LangGraph

Project Report

NAME : SALVA SOUMYA

REGISTER NUMBER : 23MID0061

COURS NAME : IBM AGENTIC AI-12 (Slot 2)

MAIL-ID : salva.soumya2023@vitstudent.ac.in

ABSTRACT

Customer support is one of the most important services provided by any software company. As the number of customers increases, manually handling every customer request becomes difficult, time-consuming, and expensive. Customers expect quick, accurate, and consistent responses regardless of the type of issue they face. Therefore, organizations are increasingly adopting Artificial Intelligence (AI) to automate customer support processes while maintaining response quality.

This project presents an **AI-Powered Customer Support Automation System** developed using **LangGraph**, **LangChain**, **Retrieval-Augmented Generation (RAG)**, **SQLite Memory**, and **Human-in-the-Loop (HITL)** concepts. The system automatically accepts customer queries, identifies the customer's intent, routes the request to the appropriate support department, retrieves relevant information from company documents, remembers previous customer interactions, requests human approval for critical operations, and generates a final verified response. The implementation consists of four specialized support agents namely **Sales**, **Technical Support**, **Billing**, and **Account Support**. Each agent is responsible for handling specific categories of customer requests. Instead of relying only on predefined responses, the system integrates Retrieval-Augmented Generation (RAG) using company knowledge base documents. This enables the application to retrieve accurate information from documents such as Company Policy, Pricing Guide, Technical Manual, and Frequently Asked Questions (FAQ).

To improve customer experience, SQLite database memory is implemented for storing customer conversation history. This allows the system to recall previous customer issues whenever requested. Furthermore, sensitive requests such as refunds, subscription cancellations, account closure requests, compensation requests, and management escalations are not processed automatically. These requests require Human-in-the-Loop approval before proceeding further. Finally, every response generated by the system is validated by a Supervisor Agent to ensure response quality before being returned to the customer.

The completed system successfully demonstrates intelligent workflow orchestration using LangGraph while combining retrieval-based AI, persistent memory, conditional routing, and human approval into a single automated customer support solution.

1. INTRODUCTION

Customer support plays a critical role in maintaining customer satisfaction and ensuring smooth business operations. Organizations receive thousands of support requests every day related to product information, technical problems, billing concerns, subscription management, and account-related issues. Traditionally, these requests are handled manually by customer support executives, which increases operational costs and response time while reducing overall efficiency.

ABC Technologies is a Software-as-a-Service (SaaS) company that provides cloud-based business management software to customers across multiple industries. Due to its growing customer base, the company experiences a high volume of daily customer support requests. Managing these requests manually has become increasingly challenging. Customers often experience delays in receiving responses, while support executives spend considerable time identifying issues, searching documentation, and forwarding requests to different departments.

To overcome these limitations, this project develops an **AI-Powered Customer Support Automation System** using LangGraph. The proposed system automates the complete customer support workflow from receiving the customer query to generating the final verified response. Instead of relying entirely on human intervention, the application intelligently classifies customer intent, routes the request to the appropriate department, retrieves relevant company information using Retrieval-Augmented Generation (RAG), stores customer conversations using SQLite memory, performs Human-in-the-Loop approval for sensitive requests, and validates every response through a Supervisor Agent before presenting it to the customer.

The overall objective of the project is to improve support efficiency, reduce manual effort, provide consistent responses, and demonstrate the capabilities of modern AI workflow orchestration using LangGraph.

2. PROBLEM STATEMENT

Customer support departments in software companies receive a large number of customer requests every day. These requests may include product inquiries, software installation problems, password reset requests, payment issues, refund requests, subscription management, and many other service-related questions.

In traditional customer support systems, support executives manually read each customer query, determine the appropriate department, search through company documentation, respond to the customer, and maintain conversation history. This manual workflow introduces several challenges including delayed response times, increased operational costs, inconsistent responses, and inefficient management of customer interactions.

Furthermore, some customer requests such as refund processing, subscription cancellation, account closure, compensation requests, and escalation to management require approval from senior personnel before action can be taken. Managing these approval workflows manually further increases complexity and delays customer response.

Another major limitation of conventional systems is the inability to remember previous customer interactions. Customers often need to repeat the same information during every conversation, resulting in poor customer experience.

Therefore, there is a need for an intelligent customer support system capable of automatically classifying customer requests, retrieving relevant information from organizational documents, remembering previous customer interactions, obtaining human approval whenever necessary, and generating accurate responses while minimizing manual intervention.

3. BUSINESS REQUIREMENTS

Based on the problem statement provided by ABC Technologies, the proposed customer support automation system must satisfy the following business requirements:

- The system must accept customer support queries through a user interface or command-line interface.
- It should automatically identify the customer's intent by analyzing the submitted query.
- The application should classify customer requests into one of the available support departments.
- The query should be routed to the appropriate support agent responsible for handling that category of request.
- The system should retrieve relevant information from company documents whenever additional information is required.
- Previous customer interactions should be stored and retrieved using persistent memory.
- High-risk customer requests should not be processed automatically and must require human approval before proceeding.
- Every generated response should be validated before being delivered to the customer.
- The application should maintain an organized workflow using LangGraph and demonstrate modular AI workflow orchestration.

The successful implementation of these requirements ensures that the system automates the majority of customer support operations while maintaining human supervision for sensitive requests.

4. PROJECT OBJECTIVES

The primary objective of this project is to develop an intelligent AI-based customer support automation system capable of handling customer requests efficiently using LangGraph. The system should minimize manual effort while improving response quality, consistency, and operational efficiency.

The specific objectives of the project are:

- To automate customer support request processing.
- To classify customer queries into the correct support department.
- To implement specialized support agents for Sales, Technical Support, Billing, and Account Management.
- To integrate Retrieval-Augmented Generation (RAG) for document-based question answering.
- To implement semantic document retrieval using HuggingFace embeddings and FAISS vector database.
- To maintain persistent customer conversation history using SQLite database.
- To recall previous customer interactions whenever requested.
- To implement Human-in-the-Loop approval for high-risk customer requests.
- To validate all generated responses using a Supervisor Agent.
- To demonstrate workflow orchestration using LangGraph StateGraph.
- To improve overall customer support efficiency while reducing manual workload.

5. FUNCTIONAL REQUIREMENTS

The developed system performs the following major functions:

5.1 Customer Query Processing

The application accepts customer queries entered by the user and creates an initial workflow state containing customer information, query details, and response variables.

5.2 Intent Classification

The system analyzes customer queries using keyword-based classification and determines the appropriate support department.

5.3 Department Routing

Based on the identified intent, the workflow routes the request to one of the available support agents.

5.4 Knowledge Retrieval

Whenever additional information is required, the system retrieves relevant content from company knowledge base documents using Retrieval-Augmented Generation.

5.5 Memory Storage

Every completed customer interaction is stored in the SQLite database for future reference.

5.6 Memory Recall

The application retrieves previous customer issues whenever the customer requests conversation history.

5.7 Human Approval

Sensitive customer requests are automatically detected and forwarded for human approval before further processing.

5.8 Supervisor Validation

Every generated response is reviewed by the Supervisor Agent before the final response is returned to the customer.

6. TECHNOLOGIES USED

The AI-Powered Customer Support Automation System was developed using a combination of modern Artificial Intelligence, Natural Language Processing, workflow orchestration, and database technologies. Each technology was selected based on its capability to solve a specific problem within the customer support workflow.

The following table summarizes the technologies used in the project.

Technology	Purpose
Python	Primary programming language used for developing the complete application.
LangGraph	Used to design and execute the workflow as a graph of interconnected nodes.
LangChain	Provides document loading, embeddings, vector search, and Retrieval-Augmented Generation (RAG).
FAISS	Vector database used for storing document embeddings and performing semantic similarity search.
HuggingFace Embeddings	Converts text documents into vector embeddings for semantic retrieval.
SQLite	Stores customer conversation history and enables memory recall.
RecursiveCharacterTextSplitter	Splits large documents into smaller chunks before embedding generation.
VS Code	Development environment used for coding, debugging, and project execution.

6.1 Python

Python was selected because of its simplicity, readability, and extensive AI ecosystem. All project modules including workflow execution, database operations, document retrieval, and customer support agents were implemented in Python.

Python also provides excellent support for libraries such as LangGraph, LangChain, FAISS, SQLite, and HuggingFace.

6.2 LangGraph

LangGraph is the core framework used in this project.

It allows the entire customer support process to be represented as a workflow graph consisting of multiple connected nodes.

Each node performs one specific task, while conditional routing determines which node executes next.

LangGraph makes the workflow modular, scalable, and easy to maintain.

6.3 LangChain

LangChain is used primarily for implementing Retrieval-Augmented Generation (RAG).

It provides utilities for:

- Loading documents
- Splitting documents into chunks
- Generating embeddings
- Connecting with FAISS
- Retrieving relevant document content

Without LangChain, implementing semantic document retrieval would require significantly more manual coding

6.4 FAISS

FAISS (Facebook AI Similarity Search) is used as the project's vector database.

After converting text chunks into vector embeddings, FAISS stores these embeddings and performs efficient similarity search.

Instead of searching documents using exact keywords, FAISS retrieves semantically similar information, producing more accurate results.

6.5 HuggingFace Embeddings

The project uses the Sentence Transformer model:

```
sentence-transformers/all-MiniLM-L6-v2
```

This model converts textual information into dense numerical vectors.

These embeddings preserve the semantic meaning of sentences, enabling accurate similarity search during RAG retrieval.

6.6 SQLite

SQLite is used to implement persistent customer memory.

Each customer interaction is stored in the database, including:

- Customer ID
- Customer Name
- Customer Query

Later, when the customer asks about a previous issue, the application retrieves the stored conversation from SQLite.

7. SYSTEM ARCHITECTURE

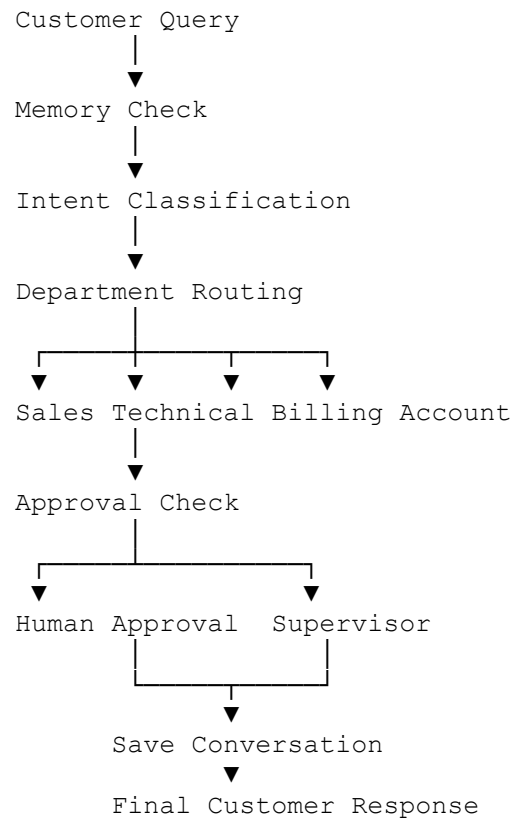
The entire application follows a modular architecture in which each processing step is represented by an independent LangGraph node.

The workflow begins when a customer submits a query. The query first passes through a memory check, followed by intent classification. Based on the identified intent, the query is routed to the appropriate support agent.

If the request is classified as a high-risk operation such as refund processing or account closure, it is forwarded to the Human Approval node before proceeding further.

Finally, every response is validated by the Supervisor Agent before the conversation is stored in SQLite and returned to the customer.

The complete workflow can be represented as follows:



8. PROJECT FOLDER STRUCTURE

The project follows a modular directory structure to improve readability and maintainability.

```
CustomerSupportAutomation/  
├── app.py  
├── graph.py  
├── state.py  
├── nodes.py  
├── agents.py  
├── approval.py  
├── supervisor.py  
├── memory.py  
├── rag.py  
├── requirements.txt  
├── README.md  
├── database/  
│   └── memory.db  
├── knowledge_base/  
│   ├── Company_Policy.txt  
│   ├── Pricing_Guide.txt  
│   ├── Technical_Manual.txt  
│   └── FAQ.txt  
└── workflow_diagram.png
```

Each file has a specific responsibility, making the application easy to understand and extend.

9. LANGGRAPH WORKFLOW IMPLEMENTATION

LangGraph is the backbone of the entire project. The workflow is created using **StateGraph**, where each processing stage is implemented as an independent node.

The workflow begins with the **Memory Check Node**. If the customer requests previous conversation history, the workflow retrieves the stored conversation directly from SQLite and bypasses intent classification. Otherwise, the query proceeds to the **Intent Classification Node**, where keywords are analyzed to determine the appropriate support department.

The classified query is then routed to one of four specialized agents:

- Sales Agent
- Technical Support Agent
- Billing Agent
- Account Agent

After the department-specific response is generated, the workflow enters the **Approval Check Node**.

- If approval is required, the request is forwarded to the Human Approval Node.
- Otherwise, it proceeds directly to the Supervisor Agent.
- The Supervisor Agent reviews the generated response and prepares the final customer response.
- Finally, the conversation is saved into SQLite before terminating the workflow.

10. STATE MANAGEMENT

LangGraph maintains a shared state object throughout the workflow.

Every node receives the same state object, modifies specific fields, and forwards it to the next node.

The implemented state contains the following fields:

Field	Description
customer_name	Name of the customer
customer_id	Unique customer identifier
query	Customer's input query
memory_response	Previous issue retrieved from SQLite
intent	Classified department
department	Assigned support department
retrieved_context	Information retrieved through RAG

Field	Description
requires_approval	Indicates whether HITL approval is required
approval_status	Approved or Rejected
agent_response	Department agent response
supervisor_response	Reviewed response generated by Supervisor
final_response	Final response delivered to customer

Using a shared state object simplifies communication between workflow nodes while ensuring consistency throughout the execution process.

11. INTENT CLASSIFICATION

The Intent Classification Node is responsible for identifying the category of each customer query.

Instead of using a machine learning classifier, the project implements keyword-based intent detection, which is sufficient for the assignment requirements.

The supported intents are:

Sales

Keywords:

- pricing
- price
- plan
- subscription
- product

Technical Support

Keywords:

- error
- crash
- login
- upload
- application
- configuration
- install

Billing

Keywords:

- billing
- payment
- refund
- invoice
- compensation
- charge
- escalate
- management

Account

Keywords:

- account
- password
- profile
- activate
- deactivate
- close
- delete

Once the intent is identified, the workflow routes the customer query to the corresponding department.

12. MULTI-AGENT ROUTING

The application implements four independent support agents.

Each agent is responsible for generating responses related to its department.

Sales Agent

The Sales Agent retrieves pricing plans, subscription details, and product information using Retrieval-Augmented Generation from the knowledge base.

Technical Support Agent

Handles application crashes, login issues, software installation problems, and configuration errors by providing troubleshooting guidance.

Billing Agent

Processes invoice requests, payment issues, billing queries, and refund-related requests.

High-risk billing operations are automatically forwarded for Human Approval.

Account Agent

Handles password reset requests, account activation, deactivation, profile updates, and account management operations.

Using multiple specialized agents improves modularity, simplifies maintenance, and closely resembles real-world customer support systems.

Excellent! This is the **final part (Part 3)** of your report. It is written according to **our implementation** and will complete your **15-page documentation**.

- Agent Routing
- RAG Retrieval
- Human Approval
- Memory Recall
- Final Response Generation

13. RETRIEVAL-AUGMENTED GENERATION (RAG) IMPLEMENTATION

13.1 Overview

One of the major features of this project is the implementation of **Retrieval-Augmented Generation (RAG)**. Instead of relying only on predefined responses, the system retrieves relevant information from company documents before generating the response.

This approach improves response accuracy because the answers are based on the company's knowledge base rather than hardcoded text. Whenever the customer asks questions related to pricing, policies, product information, or technical documentation, the system searches the uploaded documents and retrieves the most relevant content.

The RAG pipeline implemented in this project consists of document loading, text chunking, embedding generation, vector storage, semantic retrieval, and response generation.

13.2 Knowledge Base Document

Four company documents were created as the knowledge base for the application.

Document	Purpose
Company_Policy.txt	Contains refund policies, account closure rules, compensation policy, escalation policy and subscription cancellation policy.
Pricing_Guide.txt	Contains Basic, Professional and Enterprise subscription plans along with pricing details.
Technical_Manual.txt	Contains installation instructions, troubleshooting steps, configuration guidance and technical support information.
FAQ.txt	Contains frequently asked customer questions regarding passwords, profile updates, billing and general product information.

13.3 Document Loading

The project uses the **TextLoader** provided by LangChain Community to load all text files from the knowledge base directory.

Instead of loading a single file, the application automatically scans the knowledge_base folder and loads every text document available. This makes the system scalable because additional documents can be added without modifying the code.

13.4 Document Chunking

Large documents cannot be embedded efficiently as a single block of text. Therefore, every document is divided into smaller chunks before embedding generation.

The project uses:

- Chunk Size = 500 characters
- Chunk Overlap = 50 characters

The RecursiveCharacterTextSplitter ensures that important contextual information is preserved between adjacent chunks while maintaining manageable chunk sizes.

13.5 Embedding Generation

After chunking, each document chunk is converted into a numerical vector representation using the HuggingFace Sentence Transformer model.

Model Used:

```
sentence-transformers/all-MiniLM-L6-v2
```

Embeddings capture the semantic meaning of text rather than relying on exact keyword matching. This enables the system to retrieve information even when the customer uses different wording.

13.6 FAISS Vector Store

Once embeddings are generated, they are stored inside a FAISS vector database.

FAISS performs similarity search by comparing the embedding of the customer query with all stored document embeddings.

The most semantically similar chunks are returned as the retrieved context.

The project retrieves the **Top 2** matching document chunks for every query.

13.7 Context Retrieval

Whenever the Sales Agent receives a pricing-related query, the `retrieve_context()` function performs the following operations:

1. Load documents.
2. Split documents into chunks.
3. Generate embeddings.
4. Create FAISS vector database.
5. Convert customer query into embedding.
6. Retrieve the most relevant document chunks.
7. Return retrieved context to the Sales Agent.

The retrieved context becomes part of the final response returned to the customer.

14. SQLITE MEMORY IMPLEMENTATION

14.1 Overview

Customer conversation history is stored using SQLite database.

Memory allows the application to remember previous customer interactions even after multiple workflow executions.

This functionality improves customer experience because customers do not need to repeat their previous issues.

14.2 Database Schema

The project creates a database named:

```
memory.db
```

Inside the database, a table named **conversations** is created.

The table contains the following fields.

Column	Description
id	Auto Increment Primary Key
customer_id	Customer Identifier
customer_name	Customer Name
query	Customer Query

14.3 Saving Customer Conversations

After every successful workflow execution, the Save Memory node stores the following information into SQLite.

- Customer ID
- Customer Name
- Customer Query

This ensures that all customer interactions are permanently stored.

14.4 Memory Recall

Whenever the customer asks

`What was my previous support issue?`

the Memory Check Node executes before Intent Classification.

The application searches SQLite and retrieves the customer's previous support query.

Example:

Current Query:

`What was my previous support issue?`

Output:

`Your previous support issue was:`

`I have a billing issue.`

This demonstrates persistent conversation memory.

15. HUMAN-IN-THE-LOOP (HITL) APPROVAL

15.1 Overview

Certain customer requests should never be approved automatically.

These requests require manual verification by a Human Supervisor.

The Human-in-the-Loop workflow ensures that sensitive operations remain under human control.

15.2 Approval Requests

The application requires approval for:

- Refund Requests
- Subscription Cancellation
- Account Closure
- Compensation Requests
- Escalation to Management

These conditions exactly follow the assignment requirements.

15.3 Approval Workflow

After the department agent generates a response, the Approval Check Node examines the customer query.

If approval is required:

```
requires_approval = True
```

the workflow is routed to the Human Approval Node.

The supervisor is prompted with:

```
Approve request? (yes/no)
```

Depending on the response:

```
Approved
```

or

```
Rejected
```

is stored in the workflow state.

16. SUPERVISOR AGENT

16.1 Purpose

The Supervisor Agent is the final quality assurance stage of the workflow.

Every response generated by the department agents passes through the Supervisor Agent before reaching the customer.

This ensures consistency, correctness, and validation of responses.

16.2 Responsibilities

The Supervisor Agent performs the following tasks:

- Reviews agent response.
- Checks approval status.
- Validates response quality.
- Generates supervisor review.
- Produces the final customer response.

If Human Approval was required, the supervisor indicates that the request has been reviewed and approved.

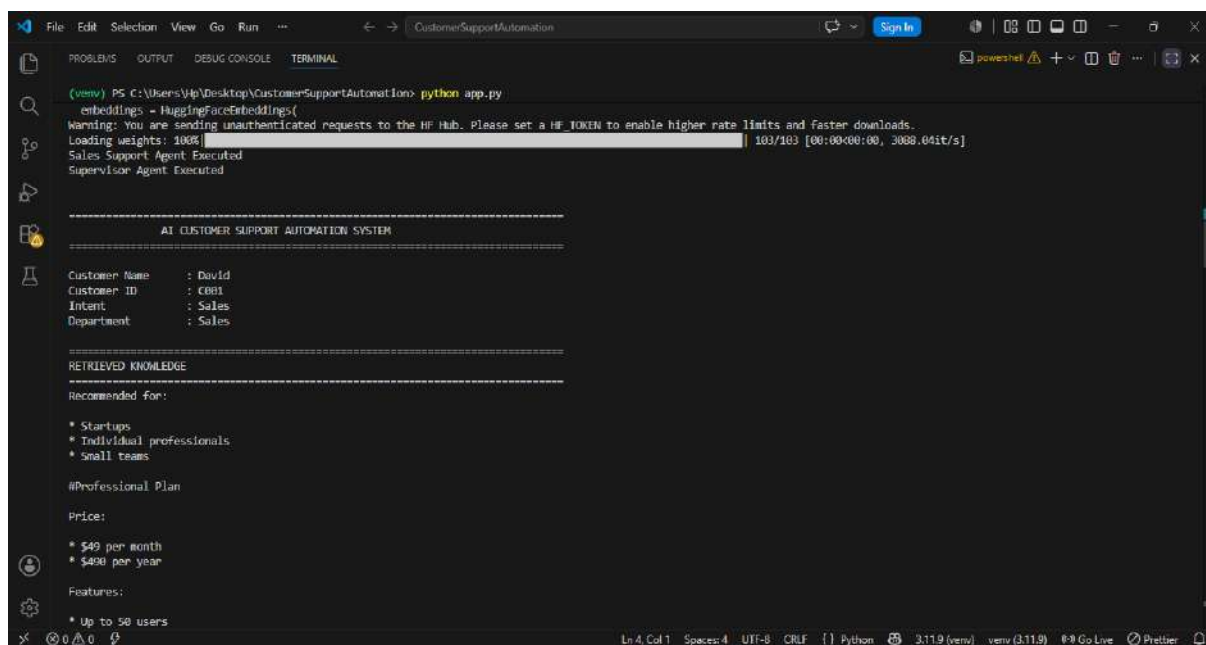
Otherwise, it confirms that the response has been verified.

17. TESTING AND RESULTS

The application was tested using the demonstration queries provided in the assignment.

QUERY -1 :

What are the pricing plans available for your software?



```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Yip\Desktop\CustomerSupportAutomation> python app.py
embeddings = HuggingFaceEmbeddings(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% | 103/103 [00:00<00:00, 3088.64it/s]
Sales Support Agent Executed
Supervisor Agent Executed

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Customer Name      : David
Customer ID       : C001
Intent            : Sales
Department        : Sales

=====
RETRIEVED KNOWLEDGE
=====
Recommended for:

* Startups
* Individual professionals
* Small teams

#Professional Plan

Price:

* $49 per month
* $490 per year

Features:

* Up to 50 users
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Features:
* Up to 50 users
* Cloud storage (200 GB)
* Priority email support
* Phone support
* Advanced analytics
* Daily backup
* API access
Recommended for:
* Small and medium businesses
#Enterprise Plan
Price:
* Custom pricing
Features:
Price:
* Custom pricing
Features:
* Unlimited users
* Unlimited cloud storage
* Dedicated account manager
* 24x7 premium support
* Custom integrations
* Advanced security
* Disaster recovery
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Recommended for:
* Large organizations
#Subscription Information
Customers can upgrade or downgrade their subscription at any time.
Annual subscriptions receive a discount compared to monthly billing.
Enterprise pricing is provided after consultation with the Sales team.
=====
AGENT RESPONSE
Sales Support Response
=====
Recommended for:
* Startups
* Individual professionals
* Small teams
#Professional Plan
Prices:
* $40 per month
* $400 per year
Features:
* Up to 50 users
* Cloud storage (200 GB)
* Priority email support
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Recommended for:
* Small and medium businesses
#Enterprise Plan
Price:
* Custom pricing
Features:
Price:
* Custom pricing
Features:
* Unlimited users
* Unlimited cloud storage
* Dedicated account manager
* 24x7 premium support
* Custom integrations
* Advanced security
* Disaster recovery
* SLA guarantee
Recommended for:
* Large organizations
#Subscription Information
Customers can upgrade or downgrade their subscription at any time.
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation> python app.py
Enterprise pricing is provided after consultation with the Sales team.
=====
AGENT RESPONSE
=====
Sales Support Response

Recommended for:
* Startups
* Individual professionals
* Small teams

#Professional Plan
Price:
* $49 per month
* $496 per year

Features:
* Up to 50 users
* Cloud storage (200 GB)
* Priority email support
* Phone support
* Advanced analytics
* Daily backup
* API access

Recommended for:
* Small and medium businesses
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation> python app.py
#Enterprise Plan
Price:
* Custom pricing

Features:
Price:
* Custom pricing

Features:
* Unlimited users
* Unlimited cloud storage
* Dedicated account manager
* 24x7 premium support
* Custom integrations
* Advanced security
* Disaster recovery
* SLA guarantee

Recommended for:
* Large organizations

#Subscription Information
Customers can upgrade or downgrade their subscription at any time.
Annual subscriptions receive a discount compared to monthly billing.
Enterprise pricing is provided after consultation with the Sales team.
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation> python app.py
=====
APPROVAL STATUS
=====
Approval Required : NO

=====
SUPERVISOR REVIEW
=====
Sales Support Response

Recommended for:
* Startups
* Individual professionals
* Small teams

#Professional Plan
Price:
* $49 per month
* $496 per year

Features:
* Up to 50 users
* Cloud storage (200 GB)
* Priority email support
* Phone support
* Advanced analytics
* Daily backup
* API access

Recommended for:
* Small and medium businesses
```

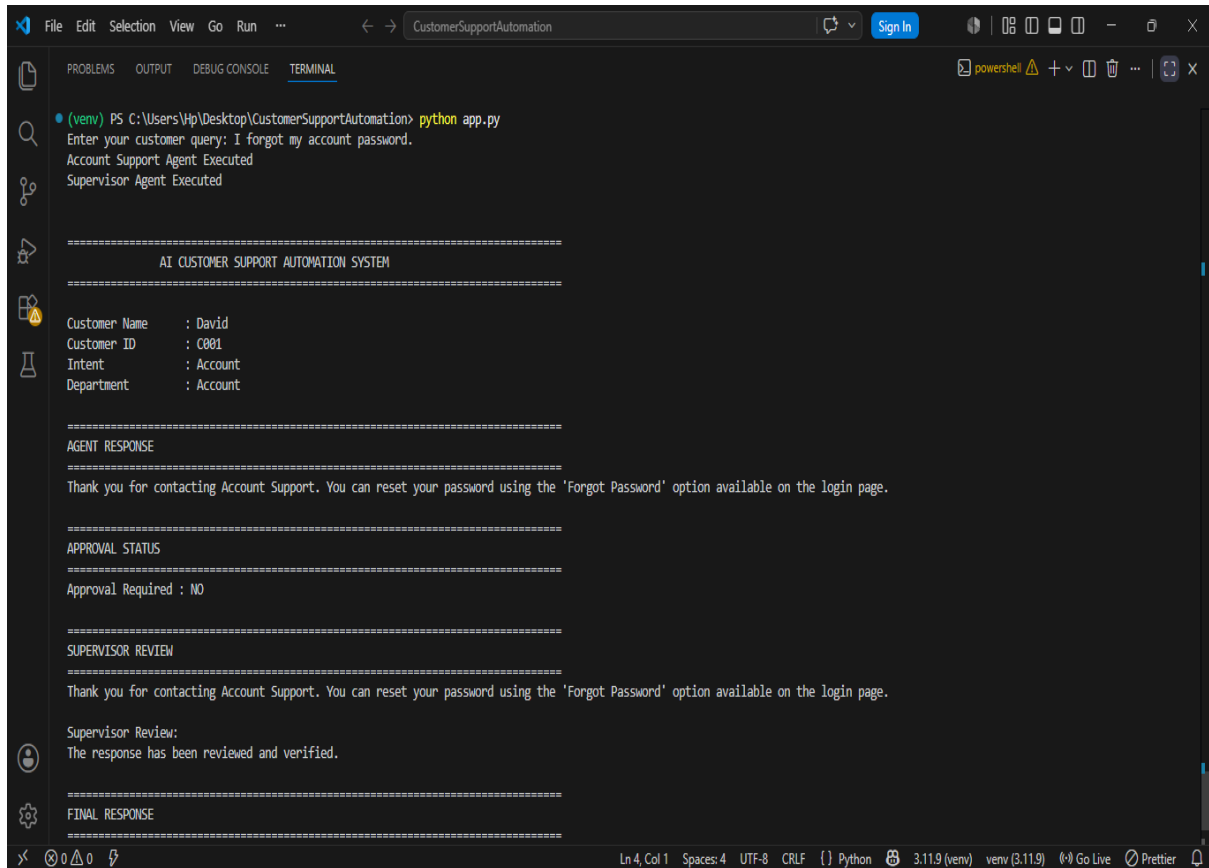
```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Recommended for:
* Small and medium businesses
#Enterprise Plan
Price:
* Custom pricing
Features:
Price:
* Custom pricing
Features:
* Unlimited users
* Unlimited cloud storage
* Dedicated account manager
* 24x7 premium support
* Custom integrations
* Advanced security
* Disaster recovery
* SLA guarantee
Recommended for:
* Large organizations
#Subscription Information
Customers can upgrade or downgrade their subscription at any time.
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
=====
FINAL RESPONSE
=====
Sales Support Response
Recommended for:
* Startups
* Traditional professionals
* Small teams
#Professional Plan
Price:
* $40 per month
* $400 per year
Features:
* Up to 50 users
* Cloud storage (200 GB)
* Priority email support
* Phone support
* Advanced analytics
* Daily backup
* API access
Recommended for:
* Small and medium businesses
#Enterprise Plan
Price:
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Features:
Price:
* Custom pricing
Features:
* Unlimited users
* Unlimited cloud storage
* Dedicated account manager
* 24x7 premium support
* Custom integrations
* Advanced security
* Disaster recovery
* SLA guarantee
Recommended for:
* Large organizations
#Subscription Information
Customers can upgrade or downgrade their subscription at any time.
Annual subscriptions receive a discount compared to monthly billing.
Enterprise pricing is provided after consultation with the Sales team.
Supervisor Review:
The response has been reviewed and verified.
=====
PROCESS COMPLETED SUCCESSFULLY
=====
```

QUERY -2 :

I forgot my account password.



```
File Edit Selection View Go Run ... CustomerSupportAutomation Sign In
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation> python app.py
Enter your customer query: I forgot my account password.
Account Support Agent Executed
Supervisor Agent Executed

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====

Customer Name      : David
Customer ID       : C081
Intent            : Account
Department        : Account

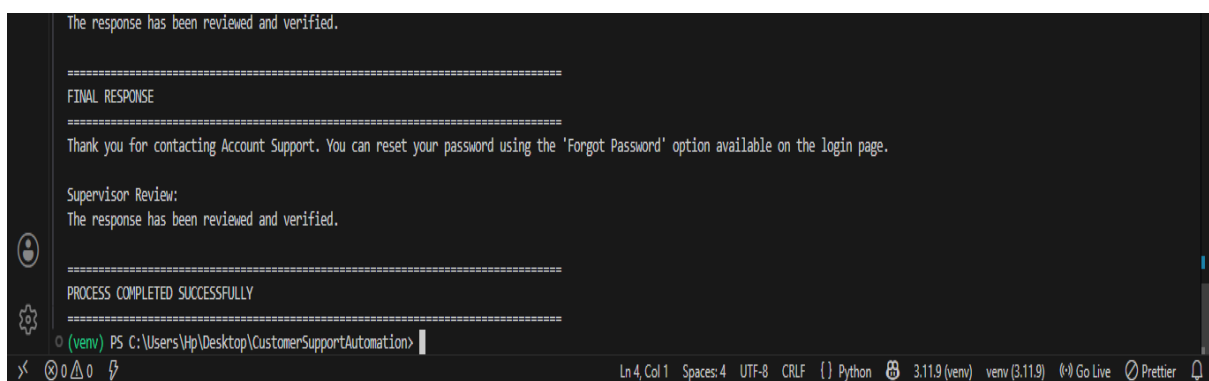
=====
AGENT RESPONSE
=====
Thank you for contacting Account Support. You can reset your password using the 'Forgot Password' option available on the login page.

=====
APPROVAL STATUS
=====
Approval Required : NO

=====
SUPERVISOR REVIEW
=====
Thank you for contacting Account Support. You can reset your password using the 'Forgot Password' option available on the login page.

Supervisor Review:
The response has been reviewed and verified.

=====
FINAL RESPONSE
=====
```



```
The response has been reviewed and verified.

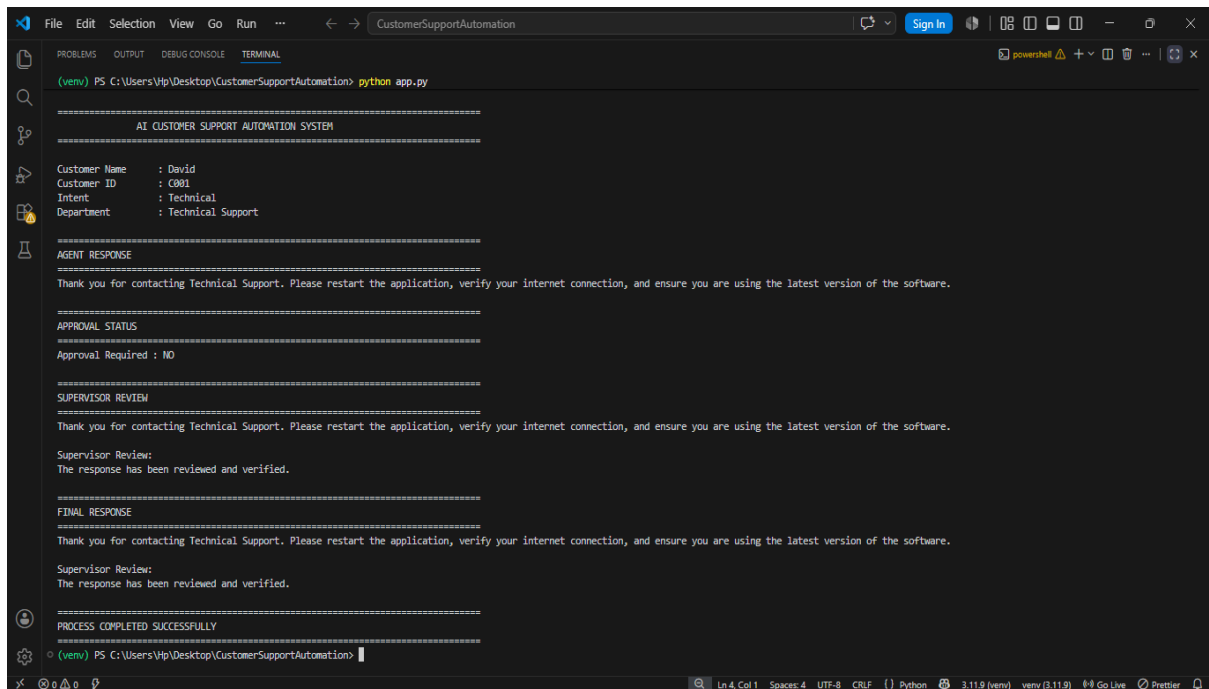
=====
FINAL RESPONSE
=====
Thank you for contacting Account Support. You can reset your password using the 'Forgot Password' option available on the login page.

Supervisor Review:
The response has been reviewed and verified.

=====
PROCESS COMPLETED SUCCESSFULLY
=====
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation>
```

QUERY – 3 :

My application crashes whenever I upload a file.

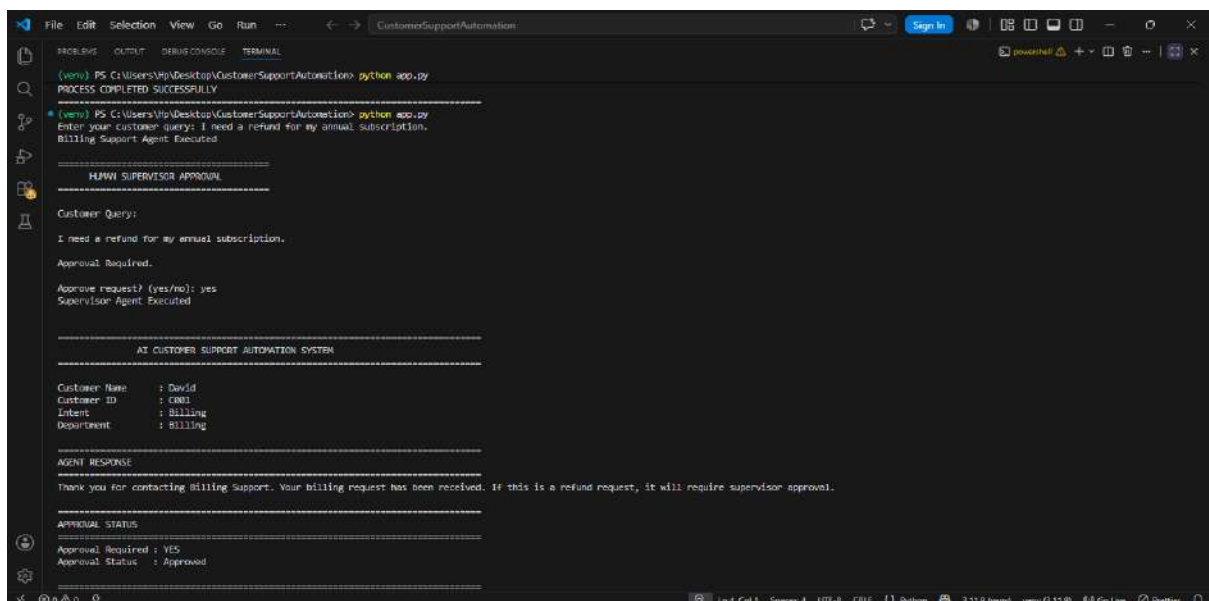


```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation> python app.py

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Customer Name      : David
Customer ID       : C001
Intent            : Technical
Department        : Technical Support
=====
AGENT RESPONSE
=====
Thank you for contacting Technical Support. Please restart the application, verify your internet connection, and ensure you are using the latest version of the software.
=====
APPROVAL STATUS
=====
Approval Required : NO
=====
SUPERVISOR REVIEW
=====
Thank you for contacting Technical Support. Please restart the application, verify your internet connection, and ensure you are using the latest version of the software.
=====
Supervisor Review:
The response has been reviewed and verified.
=====
FINAL RESPONSE
=====
Thank you for contacting Technical Support. Please restart the application, verify your internet connection, and ensure you are using the latest version of the software.
=====
Supervisor Review:
The response has been reviewed and verified.
=====
PROCESS COMPLETED SUCCESSFULLY
=====
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation>
```

QUERY -4 :

I need a refund for my annual subscription.



```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation> python app.py
PROCESS COMPLETED SUCCESSFULLY
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation> python app.py
Enter your customer query: I need a refund for my annual subscription.
Billing Support Agent Executed

=====
HUMAN SUPERVISOR APPROVAL
=====
Customer Query:
I need a refund for my annual subscription.
Approval Required:
Approve request? (yes/no): yes
Supervisor Agent Executed

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Customer Name      : David
Customer ID       : C001
Intent            : Billing
Department        : Billing
=====
AGENT RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.
=====
APPROVAL STATUS
=====
Approval Required : YES
Approval Status   : Approved
=====
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation>
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation Sign In
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation> python app.py

=====
SUPERVISOR REVIEW
=====
Thank you for contacting Technical Support. Please restart the application, verify your internet connection, and ensure you are using the latest version of the software.

Supervisor Review:
The response has been reviewed and verified.

=====
FINAL RESPONSE
=====
Thank you for contacting Technical Support. Please restart the application, verify your internet connection, and ensure you are using the latest version of the software.

Supervisor Review:
The response has been reviewed and verified.

=====
PROCESS COMPLETED SUCCESSFULLY
=====
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation> python app.py
Enter your customer query: I need a refund for my annual subscription.
Billing Support Agent Executed

=====
HUMAN SUPERVISOR APPROVAL
=====
Customer Query:

I need a refund for my annual subscription.

Approval Required.

Approve request? (yes/no): yes
Supervisor Agent Executed

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Ln 4, Col 1  Spaces: 4  UTF-8  CRLF  Python  3.11.9 (venv)  venv (3.11.9)  Go Live  Prettier
```

```
File Edit Selection View Go Run ... CustomerSupportAutomation Sign In
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation> python app.py

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Customer Name      : David
Customer ID       : 0001
Intent            : Billing
Department        : Billing

=====
AGENT RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

=====
APPROVAL STATUS
=====
Approval Required : YES
Approval Status   : Approved

=====
SUPERVISOR REVIEW
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

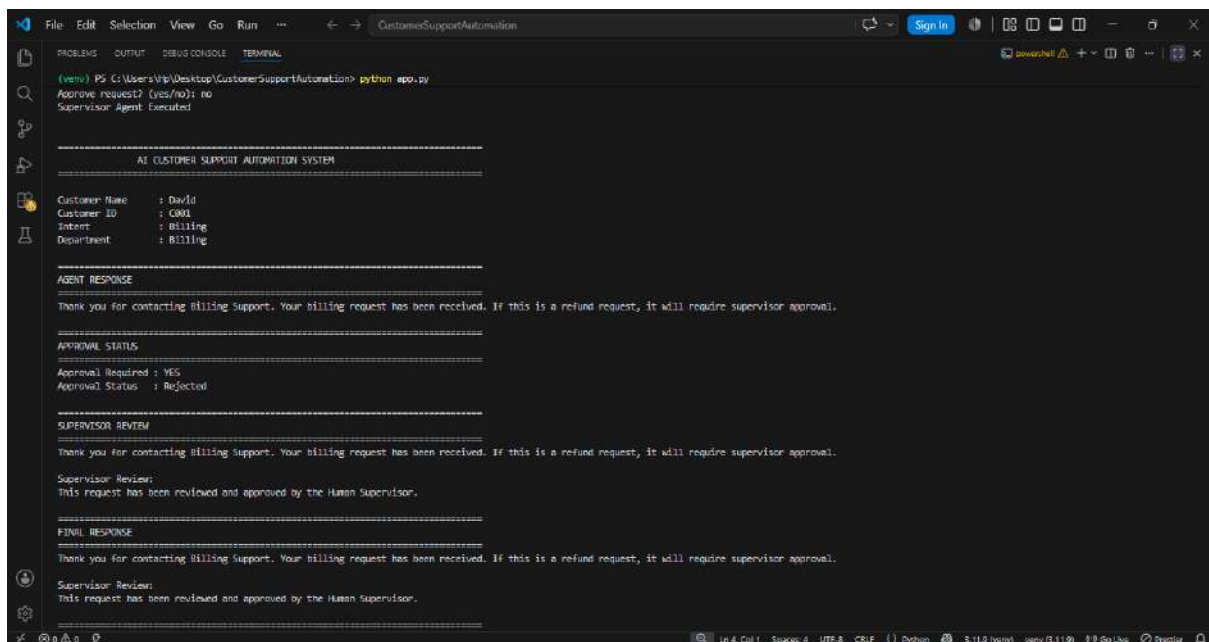
Supervisor Review:
This request has been reviewed and approved by the Human Supervisor.

=====
FINAL RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

Supervisor Review:
This request has been reviewed and approved by the Human Supervisor.

=====
PROCESS COMPLETED SUCCESSFULLY
=====
(venv) PS C:\Users\Vip\Desktop\CustomerSupportAutomation>
```


When approve request is : no



```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Approve request? (yes/no): no
Supervisor Agent Executed

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Customer Name : David
Customer ID : C001
Intent : Billing
Department : Billing

=====
AGENT RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

=====
APPROVAL STATUS
=====
Approval Required : YES
Approval Status : Rejected

=====
SUPERVISOR REVIEW
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

Supervisor Review:
This request has been reviewed and approved by the Human Supervisor.

=====
FINAL RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

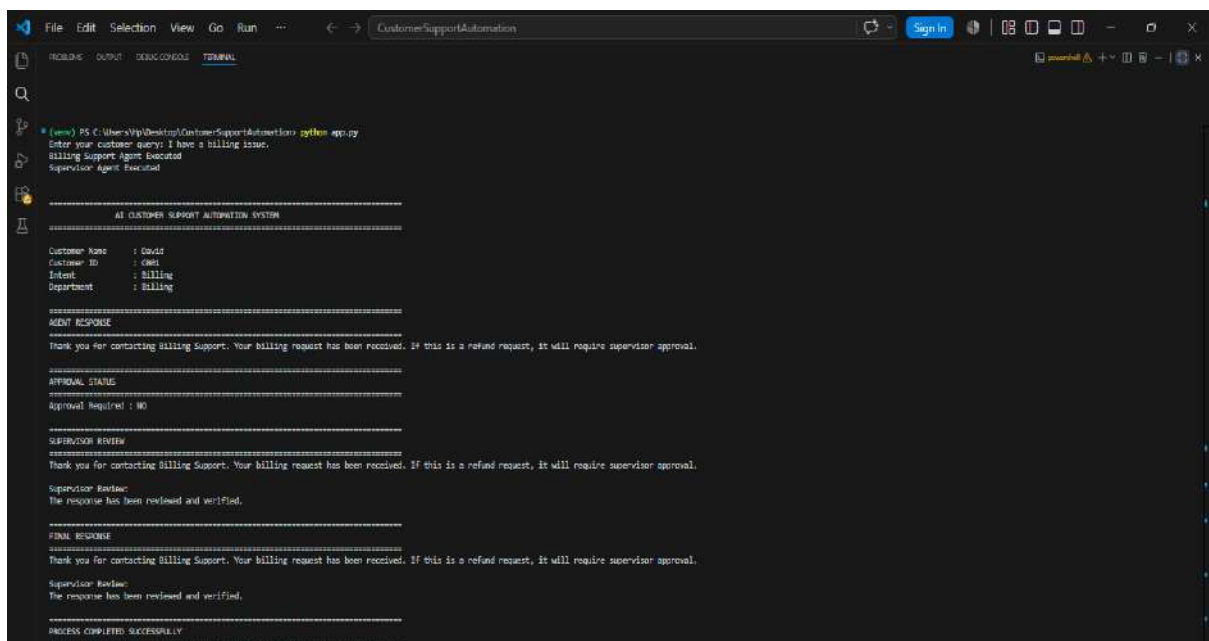
Supervisor Review:
This request has been reviewed and approved by the Human Supervisor.

=====
```

QUERY -5 :

I have a billing issue.

What was my previous support issue?



```
File Edit Selection View Go Run ... CustomerSupportAutomation
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
(venv) PS C:\Users\vip\Desktop\CustomerSupportAutomation> python app.py
Enter your customer query: I have a billing issue.
Billing Support Agent Executed
Supervisor Agent Executed

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====
Customer Name : David
Customer ID : C001
Intent : Billing
Department : Billing

=====
AGENT RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

=====
APPROVAL STATUS
=====
Approval Required : NO

=====
SUPERVISOR REVIEW
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

Supervisor Review:
The response has been reviewed and verified.

=====
FINAL RESPONSE
=====
Thank you for contacting Billing Support. Your billing request has been received. If this is a refund request, it will require supervisor approval.

Supervisor Review:
The response has been reviewed and verified.

=====
PROCESS COMPLETED SUCCESSFULLY
=====
```

```
(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation> python app.py
Enter your customer query: What was my previous support issue?

=====
AI CUSTOMER SUPPORT AUTOMATION SYSTEM
=====

Customer Name      : David
Customer ID       : 0001
Intent            : Memory
Department        :

=====

MEMORY RECALL
=====
Your previous support issue was: I have a billing issue.

=====

APPROVAL STATUS
=====
Approval Required : NO

=====

PROCESS COMPLETED SUCCESSFULLY
=====

(venv) PS C:\Users\Hp\Desktop\CustomerSupportAutomation>
```

18. ADVANTAGES

The proposed customer support automation system provides several advantages over traditional customer support systems.

- Automates customer support workflows.
- Reduces manual intervention.
- Improves response consistency.
- Supports intelligent document retrieval.
- Maintains customer conversation history.
- Implements secure Human-in-the-Loop approval.
- Uses modular LangGraph architecture.
- Easy to maintain and extend.
- Demonstrates real-world AI workflow orchestration.

19. LIMITATIONS

Although the developed system successfully satisfies the assignment requirements, certain limitations exist.

- Intent classification uses keyword-based logic instead of machine learning.
- Only text-based knowledge base documents are supported.
- Human approval is simulated using terminal input.
- No graphical web interface is available.
- Authentication and user management are not implemented.

These limitations can be addressed in future versions.

20. FUTURE ENHANCEMENTS

Several enhancements can further improve the project.

- Integration with Large Language Models (LLMs) such as GPT or Llama.
- Deployment as a web application using Streamlit or FastAPI.
- Voice-enabled customer support.
- Multi-language query support.
- Email integration.
- CRM integration.
- Customer authentication.
- Cloud deployment.
- Analytics dashboard.
- AI-powered intent classification using transformer models.

21. CONCLUSION

The AI-Powered Customer Support Automation System successfully demonstrates how workflow orchestration, document retrieval, persistent memory, and human supervision can be integrated into a single intelligent application.

The project satisfies all the business requirements specified by ABC Technologies. Customer queries are automatically classified, routed to specialized department agents, enriched using Retrieval-Augmented Generation, stored using SQLite memory, reviewed through Human-in-the-Loop approval whenever required, and validated by a Supervisor Agent before producing the final customer response.

The modular implementation using LangGraph allows each component to operate independently while sharing a common workflow state. This architecture makes the application scalable, maintainable, and suitable for real-world customer support automation.

Overall, the project demonstrates practical implementation of modern AI concepts including workflow orchestration, semantic document retrieval, persistent conversation memory, conditional routing, and human oversight, resulting in an efficient, reliable, and intelligent customer support system.

22. REFERENCES

1. LangGraph Documentation – <https://langchain-ai.github.io/langgraph/>
2. LangChain Documentation – <https://python.langchain.com/>
3. Hugging Face Sentence Transformers – <https://huggingface.co/sentence-transformers>
4. FAISS Documentation – <https://faiss.ai/>
5. SQLite Documentation – <https://www.sqlite.org/>
6. Python Documentation – <https://docs.python.org/3/>
7. ABC Technologies Assignment Specification (Provided Problem Statement)